

kanban-kit

Selbst-hostbares Kanban-Board. Entwickelt mit KI. Geführt von einem Menschen.

Ich habe kanban-kit gebaut, um zu zeigen, was KI-gestützte Entwicklung leisten kann, wenn harte Leitplanken den Takt vorgeben. Jede Zeile ist durch Tests, Mutationstests und statische Analysen abgesichert. Wenn eine Regel verletzt wird, bricht der Build. Das Ergebnis ist kein Prototyp, sondern ein Produkt im täglichen Einsatz.

~95.800

Zeilen im Repository,
geschrieben von der KI,
verantwortet von mir

> 2.600

Tests, davon 375
Integrationstests gegen
echtes Postgres und
MinIO

100 %

Line- und Branch-
Coverage, Backend und
Frontend, als hartes
Build-Gate

1.308 / 1.308

Mutationen getötet: 100
% Mutation Score (PIT)

CODEBASIS

Backend (Java)

Produktivcode	351 Klassen , ~18.400 Zeilen
Architektur	9 Fachmodule, hexagonal geschnitten
Testcode	~36.800 Zeilen , das Doppelte des Produktivcodes
Testläufe	1.509 (1.134 Unit/Slice, 375 Integration via Testcontainers)

Frontend (TypeScript/React)

Produktivcode	99 Dateien , ~13.100 Zeilen
Testcode	~17.200 Zeilen , auch hier mehr Test als Code
Tests	1.184 (Vitest und Testing Library)
Coverage	100 % Line und Branch (v8), Build- Gate

Dazu Datenbank-Migrationen, Build- und Infrastruktur-Konfiguration, Werkzeug-Skripte und Dokumentation. Zusammen ~95.800 Zeilen im Repository.

LEITPLANKEN

Qualität, gemessen

Mutationstests	1.308 von 1.308 Mutationen getötet (PIT)
SonarCloud	Quality Gate grün, 0 Bugs, 0 Vulnerabilities , 0 % Duplikate

Regeln, die den Build brechen

Statisch	ArchUnit, Checkstyle, PMD, SpotBugs, Error Prone, NullAway/ JSpecify, Spotless
Prinzip	Leitplanken argumentieren nicht. Sie scheitern.

Wer KI unkontrolliert arbeiten lässt, produziert schneller schlechten Code. Wer gelernt hat, sie zu führen, arbeitet schneller und besser. kanban-kit ist mein Beleg dafür.